

Types of Databases

Chapter 2 | PostgreSQL Complete Guide Difficulty: Beginner | Estimated Reading Time: 25 minutes

Table of Contents

- [Learning Objectives](#)
- [Theory: Database Classification](#)
- [Relational Databases \(RDBMS\)](#)
- [NoSQL Databases](#)
- [NewSQL Databases](#)
- [Other Specialized Databases](#)
- [ASCII Visual Diagrams](#)
- [Practical Examples](#)
- [PostgreSQL Commands](#)
- [SQL Examples](#)
- [Common Mistakes](#)
- [Best Practices](#)
- [Performance Considerations](#)
- [Interview Questions](#)
- [Interview Answers](#)
- [Hands-on Exercises](#)
- [Solutions](#)
- [Advanced Notes](#)
- [Cross-References](#)

Learning Objectives

By the end of this chapter, you will be able to:

- Classify databases by data model, purpose, and storage mechanism
- Compare relational vs. NoSQL databases and identify when to use each
- Understand the trade-offs between different NoSQL paradigms (document, key-value, column, graph)
- Explain the emergence of NewSQL databases and their advantages
- Choose the right database type for a given business problem

Theory: Database Classification

Databases can be classified along several dimensions:

Classification by Data Model

Category	Examples	Best For
Relational (RDBMS)	PostgreSQL, MySQL, Oracle, SQL Server	Structured data, complex queries
Document	MongoDB, CouchDB, Firestore	Semi-structured, nested data
Key-Value	Redis, DynamoDB, Riak	Caching, sessions, simple lookups

Wide-Column	Cassandra, HBase, Bigtable	Time-series, write-heavy workloads
Graph	Neo4j, Amazon Neptune, ArangoDB	Relationship-heavy data, social networks
Time-Series	InfluxDB, TimescaleDB, Prometheus	IoT, metrics, financial ticks
Search	Elasticsearch, Solr, OpenSearch	Full-text search, log analytics
NewSQL	CockroachDB, Google Spanner, TiDB	ACID at global scale
In-Memory	Redis, Memcached, VoltDB	Ultra-low latency
Spatial	PostGIS (PostgreSQL extension)	GIS, location data
Multi-Model	ArangoDB, CosmosDB, PostgreSQL	Multiple access patterns

Classification by Location

- **On-Premises:** Deployed on company hardware
- **Cloud-Managed:** AWS RDS, Azure SQL, Google Cloud SQL
- **Serverless:** Neon, PlanetScale, Aurora Serverless
- **Edge/Embedded:** SQLite, DuckDB

Classification by Purpose

- **OLTP (Online Transaction Processing):** Day-to-day operations
- **OLAP (Online Analytical Processing):** Data warehousing, analytics
- **HTAP (Hybrid Transactional/Analytical Processing):** Both simultaneously

Relational Databases (RDBMS)

The relational model, introduced by E.F. Codd in 1970, organizes data into **tables** (relations) consisting of **rows** (tuples) and **columns** (attributes).

Core Characteristics

- **Schema-on-write:** Structure is defined before data is inserted
- **ACID transactions:** Guarantees data integrity
- **SQL:** Standardized query language
- **Joins:** Data across tables can be combined
- **Referential integrity:** Foreign keys enforce relationships

Major RDBMS Products

System	Organization	License	Specialty
PostgreSQL	Open Source Community	Open Source	Full-featured, extensible
MySQL	Oracle	Open Source / Commercial	Web applications
MariaDB	MariaDB Foundation	Open Source	MySQL fork

Oracle Database	Oracle Corp	Commercial	Enterprise, large-scale
SQL Server	Microsoft	Commercial	Windows enterprise
SQLite	D. Richard Hipp	Public Domain	Embedded, lightweight
IBM Db2	IBM	Commercial	Mainframe, enterprise

When to Use RDBMS

- Data is structured and relationships are well-defined
- ACID compliance is required (banking, healthcare, e-commerce)
- Complex queries involving multiple tables are needed
- Data integrity and consistency are critical
- Compliance requirements mandate auditable transactions

NoSQL Databases

NoSQL ("Not Only SQL") databases emerged in the 2000s to address the scalability limitations of traditional RDBMS when dealing with massive, distributed workloads.

1. Document Databases

Store data as JSON-like documents. Each document can have a different structure (schema-less).

```
// MongoDB document example
{
  "_id": "507f1f77bcf86cd799439011",
  "name": "Alice Johnson",
  "email": "alice@example.com",
  "address": {
    "street": "123 Main St",
    "city": "New York",
    "zip": "10001"
  },
  "orders": [
    {"order_id": 1, "amount": 59.99},
    {"order_id": 2, "amount": 129.50}
  ]
}
```

Best for: Content management, product catalogs, user profiles.

2. Key-Value Databases

The simplest model: a hash map at massive scale.

Key	Value
-----	-----
user:1001	--> {"name": "Alice", "age": 30}

```
session:abc --> {"user_id": 1001, "expires": ...}
cart:1001    --> [item1, item2, item3]
```

Best for: Caching, session storage, leaderboards, real-time counters.

3. Wide-Column (Column-Family) Databases

Store data in columns grouped into column families. Optimized for sparse data and write-heavy workloads.

Best for: IoT sensor data, time-series, analytics with known access patterns.

4. Graph Databases

Store nodes (entities) and edges (relationships). Optimized for traversing complex relationships.

Best for: Social networks, recommendation engines, fraud detection, knowledge graphs.

NewSQL Databases

NewSQL databases combine the scalability of NoSQL with the ACID guarantees of traditional RDBMS.

Feature	RDBMS	NoSQL	NewSQL
ACID Transactions	Yes	Usually No	Yes
Horizontal Scaling	Difficult	Yes	Yes
SQL Interface	Yes	No/Limited	Yes
Schema	Strict	Flexible	Flexible
Global Distribution	No	Yes	Yes

Examples: CockroachDB, Google Spanner, TiDB, YugabyteDB, PlanetScale.

Other Specialized Databases

Time-Series Databases

Optimized for sequential, time-stamped data:

- InfluxDB — IoT metrics, application monitoring
- TimescaleDB — PostgreSQL extension for time-series
- Prometheus — metrics for Kubernetes/cloud-native

Search Engines

Full-text search with relevance ranking:

- Elasticsearch — log analytics, search-as-a-service
- Apache Solr — enterprise search

In-Memory Databases

All data stored in RAM for microsecond latency:

- Redis — caching, pub/sub, distributed locks
- Memcached — simple distributed caching

ASCII Visual Diagrams

Diagram 1: Database Type Landscape

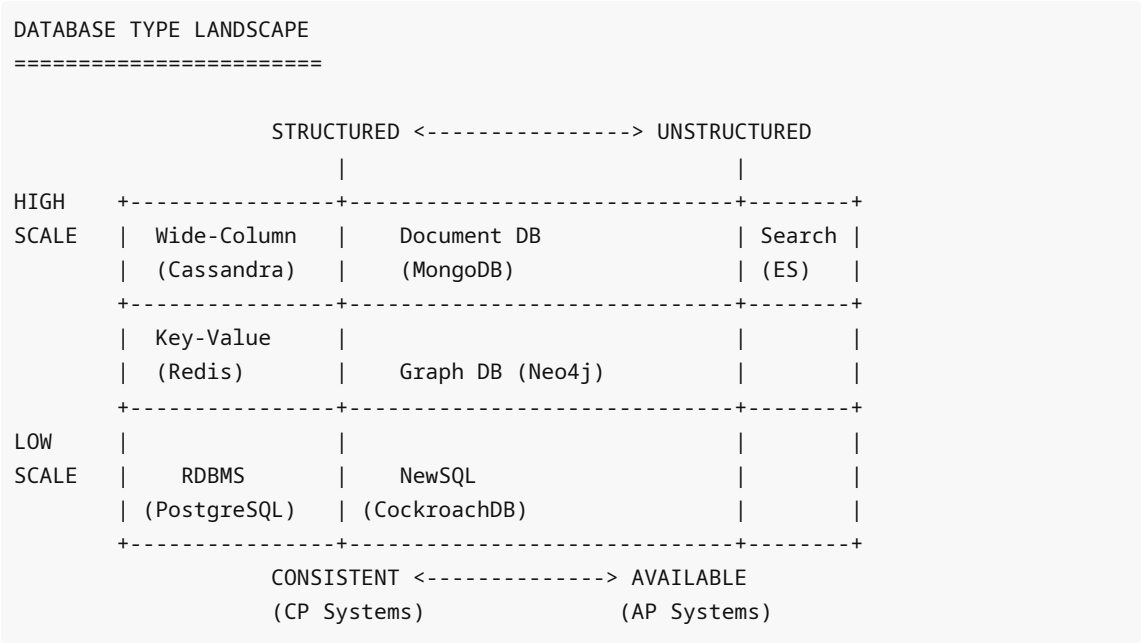


Diagram 2: NoSQL Data Models Compared

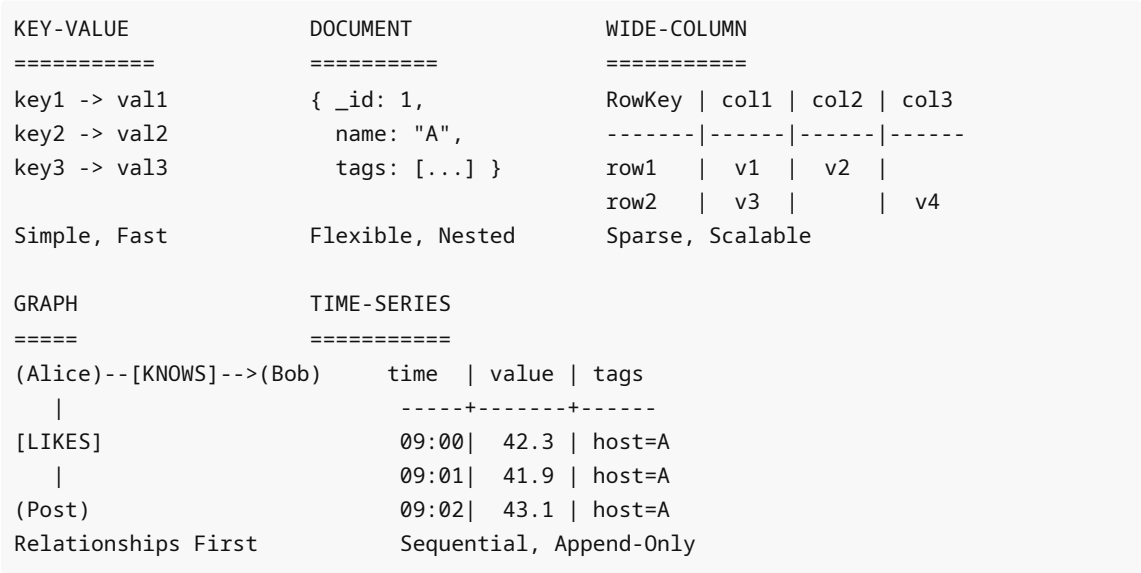
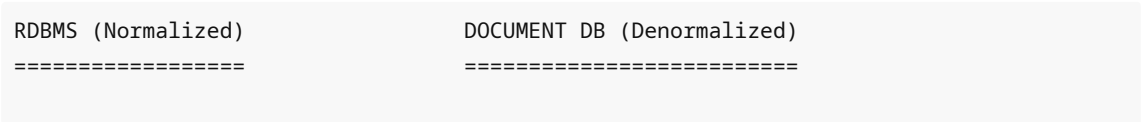


Diagram 3: RDBMS vs. Document DB Data Model



customers	{
+-----+-----+-----+	"customer_id": 1,
id name city	"name": "Alice",
+-----+-----+-----+	"city": "NY",
1 Alice NY	"orders": [
+-----+-----+-----+	{ "id": 101, "total": 50.00,
	"items": [...] },
orders	]
+-----+-----+-----+	}
id c_id total	
+-----+-----+-----+	One read = full customer picture
101 1 50.00	BUT: harder to query across customers
+-----+-----+-----+	
JOIN needed to combine Multiple tables	Embedded for locality, but duplicates data

Practical Examples

Example 1: Choosing Between PostgreSQL and MongoDB

Scenario: You are building an e-commerce product catalog.

- Products have varying attributes (a TV has resolution, refresh rate; a T-shirt has size, color)
- You need to search products by price, category, and attributes
- Orders must be transactional (charge card and reduce inventory atomically)

Analysis:

- Product catalog: MongoDB could work (flexible schema per product type)
- Order processing: PostgreSQL is better (ACID transactions required)
- **Common choice:** PostgreSQL with JSONB columns for flexible attributes, or a hybrid approach

Example 2: Redis for Session Storage

```

User logs in
  |
  v
Application creates session
  |
  v
SET session:user123 "{user_id: 123, role: admin}" EX 3600
  |
  v
On each request: GET session:user123
  |
  v
Redis returns in < 1ms (vs 10ms+ for database query)

```

Example 3: Graph Database for Fraud Detection

Traditional RDBMS query (recursive joins):
5+ table joins, complex CTEs, slow for deep traversal

Graph DB query:
MATCH (account1)-[:SENT_TO*2..5]->(account2)
WHERE account2.flagged = true
RETURN account1

Finds multi-hop money laundering in milliseconds.

PostgreSQL Commands

PostgreSQL itself can act as multiple database types with extensions:

```
-- PostgreSQL as a Document DB (JSONB)
CREATE TABLE products (
  id      SERIAL PRIMARY KEY,
  data    JSONB NOT NULL
);

INSERT INTO products (data) VALUES
('{"name": "Laptop", "specs": {"ram": "16GB", "storage": "512GB"}, "price": 999.99}');

-- Query JSONB like a document database
SELECT data->>'name' AS product_name,
       data->'specs'->>'ram' AS ram
FROM products
WHERE (data->>'price')::DECIMAL > 500;

-- PostgreSQL as a Key-Value store (hstore)
CREATE EXTENSION IF NOT EXISTS hstore;
CREATE TABLE settings (
  id      SERIAL PRIMARY KEY,
  kv      hstore
);
INSERT INTO settings (kv) VALUES ('theme => dark, lang => en, tz => UTC');
SELECT kv->'theme' FROM settings WHERE id = 1;

-- PostgreSQL as a Time-Series DB (with TimescaleDB extension)
-- CREATE EXTENSION timescaledb;
-- SELECT create_hypertable('metrics', 'time');

-- PostgreSQL as a Search Engine (full-text search)
SELECT title
FROM articles
WHERE to_tsvector('english', content) @@ to_tsquery('postgresql & database');
```

SQL Examples

Example 1: JSONB — Document-style Queries in PostgreSQL

```
-- Create a flexible product table
CREATE TABLE catalog (
  id      SERIAL PRIMARY KEY,
  sku     VARCHAR(50) UNIQUE,
  attrs   JSONB
);

-- Insert products with different schemas
INSERT INTO catalog (sku, attrs) VALUES
('TV-001', '{"type": "TV", "size": 55, "resolution": "4K", "brand": "Samsung"}'),
('SHIRT-01', '{"type": "Shirt", "size": "M", "color": "Blue", "brand": "Levis"}'),
('BOOK-001', '{"type": "Book", "title": "PostgreSQL Guide", "pages": 450}');
```

Example 2: Query JSONB

```
-- Find all TVs
SELECT sku, attrs->>'brand' AS brand
FROM catalog
WHERE attrs->>'type' = 'TV';

-- Find items with size as a number greater than 40
SELECT sku FROM catalog
WHERE (attrs->>'size')::INT > 40;
```

Example 3: Index JSONB for Performance

```
-- GIN index for fast JSONB queries
CREATE INDEX idx_catalog_attrs ON catalog USING GIN(attrs);

-- Now this query uses the index
SELECT * FROM catalog WHERE attrs @> '{"brand": "Samsung"}';
```

Example 4: Array Type (like NoSQL arrays)

```
-- PostgreSQL arrays
CREATE TABLE post (
  id      SERIAL PRIMARY KEY,
  tags    TEXT[]
);

INSERT INTO post (tags) VALUES (ARRAY['postgresql', 'database', 'sql']);
SELECT * FROM post WHERE 'postgresql' = ANY(tags);
```


Example 5: Graph-like Queries with Recursive CTEs

```
-- Simulate graph traversal
CREATE TABLE connections (
    from_user INT,
    to_user   INT
);
INSERT INTO connections VALUES (1,2),(2,3),(3,4),(4,5);

-- Find all users reachable from user 1 (up to 5 hops)
WITH RECURSIVE reach AS (
    SELECT to_user FROM connections WHERE from_user = 1
    UNION
    SELECT c.to_user FROM connections c JOIN reach r ON c.from_user = r.to_user
)
SELECT * FROM reach;
```

Example 6: Time-Series Pattern in PostgreSQL

```
CREATE TABLE metrics (
    recorded_at TIMESTAMPTZ DEFAULT NOW(),
    host        VARCHAR(50),
    cpu_pct     DECIMAL(5,2)
);

-- Query last hour's averages per 5-minute bucket
SELECT
    date_trunc('minute', recorded_at) + INTERVAL '5 min' * (EXTRACT(MINUTE FROM
recorded_at)::INT / 5) AS bucket,
    AVG(cpu_pct) AS avg_cpu
FROM metrics
WHERE recorded_at > NOW() - INTERVAL '1 hour'
GROUP BY bucket
ORDER BY bucket;
```

Example 7: Full-Text Search

```
CREATE TABLE articles (
    id        SERIAL PRIMARY KEY,
    title     TEXT,
    body      TEXT,
    tsv       TSVECTOR GENERATED ALWAYS AS (to_tsvector('english', title || ' ' ||
body)) STORED
);

CREATE INDEX idx_articles_tsv ON articles USING GIN(tsv);

SELECT title, ts_rank(tsv, query) AS rank
FROM articles, to_tsquery('english', 'postgresql & performance') query
```

```
WHERE tsv @@ query
ORDER BY rank DESC;
```

Example 8: Lateral Join (Analytics Pattern)

```
-- Get last 3 orders per customer
SELECT c.customer_id, c.name, o.order_id, o.total
FROM customers c
CROSS JOIN LATERAL (
    SELECT order_id, total
    FROM orders
    WHERE customer_id = c.customer_id
    ORDER BY order_date DESC
    LIMIT 3
) o;
```

Example 9: Compare DB Types via pg_stat_statements

```
-- Enable query statistics (requires superuser)
CREATE EXTENSION IF NOT EXISTS pg_stat_statements;

-- View top queries by total time
SELECT query, calls, total_exec_time, mean_exec_time
FROM pg_stat_statements
ORDER BY total_exec_time DESC
LIMIT 10;
```

Example 10: Check Installed Extensions

```
-- See what extensions are available
SELECT name, default_version, comment
FROM pg_available_extensions
ORDER BY name;

-- See what's installed
SELECT extname, extversion FROM pg_extension;
```

Common Mistakes

Mistake 1: Using NoSQL "Because It's Scalable"

NoSQL databases have trade-offs. Many systems start with MongoDB and later migrate to PostgreSQL because:

- Lack of transactions causes data inconsistency
- Ad-hoc queries become very complex
- Joins are done in application code, which is slower

Fix: Choose the database type based on your data model and access patterns, not hype.

Mistake 2: Assuming NoSQL = No Schema

Document databases are schema-less at the database level, but your application code always has an implicit schema. Missing schema validation leads to dirty, inconsistent data.

Mistake 3: Using RDBMS for Everything

Redis is dramatically better than PostgreSQL for caching. Elasticsearch is better for full-text search. Don't force RDBMS when a specialized tool is more appropriate.

Mistake 4: Ignoring the CAP Theorem

Different databases make different trade-offs between Consistency, Availability, and Partition Tolerance. See [Chapter 5](#).

Mistake 5: Not Considering Query Patterns First

Your query patterns should drive your data model choice, not the reverse. Ask: "What does my application need to query?" before choosing a database.

Mistake 6: Thinking PostgreSQL Can't Do NoSQL

PostgreSQL with JSONB, hstore, arrays, and extensions like TimescaleDB can handle many use cases typically assigned to NoSQL databases, with the added benefit of ACID transactions.

Best Practices

1. **Match the data model to the problem:** Use relational for structured data with complex relationships; use document stores for flexible schemas; use key-value for caching.
 2. **Don't be afraid of polyglot persistence:** Large systems often use multiple database types. PostgreSQL for transactions, Redis for caching, Elasticsearch for search.
 3. **Prefer PostgreSQL as your default:** PostgreSQL's JSONB support means you can start relational and add flexibility without switching databases.
 4. **Evaluate consistency requirements:** If your data needs ACID transactions, eliminate AP-first NoSQL databases from consideration.
 5. **Consider operational complexity:** Running Cassandra + MongoDB + Redis + PostgreSQL is four systems to monitor, patch, and tune. Consolidate where possible.
 6. **Use PostgreSQL extensions before adding new databases:** TimescaleDB, PostGIS, and pg_vector can replace InfluxDB, specialized GIS databases, and vector databases respectively.
-

Performance Considerations

- **RDBMS:** Excels at complex queries with indexes, but sharding/horizontal scaling is complex
- **Document DB:** Fast for read-heavy workloads with denormalized data; slow for cross-document queries
- **Key-Value:** Sub-millisecond reads/writes; data must fit in RAM for Redis
- **Wide-Column:** Extremely high write throughput; reads are fast only on partition key

- **Graph:** Fast for traversals; poor for aggregate queries across all nodes
-

Interview Questions

1. What is the CAP theorem and how does it relate to database selection?
 2. When would you choose MongoDB over PostgreSQL?
 3. What is polyglot persistence?
 4. What is the difference between OLTP and OLAP databases?
 5. How does PostgreSQL support document-style storage?
 6. What are the trade-offs of denormalization in document databases?
 7. What makes NewSQL different from traditional RDBMS?
 8. Why are graph databases better than RDBMS for relationship traversal?
-

Interview Answers

Q1: CAP Theorem and Database Selection

The CAP theorem states that a distributed system can only guarantee two of: Consistency (all nodes see the same data), Availability (every request gets a response), and Partition Tolerance (system continues despite network failures). In practice, partition tolerance is non-negotiable in distributed systems, so the choice is between CP (consistent but may be unavailable during partitions — e.g., HBase, Zookeeper) and AP (available but may return stale data — e.g., Cassandra, DynamoDB).

Q2: MongoDB over PostgreSQL

Choose MongoDB when: your data has a highly variable or nested structure (e.g., product catalogs), you need to iterate schema rapidly in early development, your team writes mostly object-oriented code and benefits from document-object mapping, and you don't need complex multi-document transactions. That said, PostgreSQL's JSONB often covers the same use cases.

Q3: Polyglot Persistence

Using multiple database technologies within a single application, each optimized for a specific access pattern. E.g., PostgreSQL for transactional data, Redis for session caching, Elasticsearch for search, and Kafka for event streaming.

Hands-on Exercises

Exercise 1: JSONB Document Store

Create a `people` table using JSONB to store person records with varying attributes. Insert 5 records with different fields and query them.

Exercise 2: Array Operations

Create a `blog_posts` table with a `tags` array column. Insert posts with different tags and write queries to find all posts tagged with 'postgresql'.

Exercise 3: Full-Text Search

Create an `articles` table. Insert articles about different database topics. Set up a tsvector index and search for articles matching 'relational database'.

Exercise 4: Recursive Graph Query

Model a simple org chart in PostgreSQL using a `employees` table with a `manager_id` self-referential foreign key. Write a recursive CTE to find all employees under a given manager.

Exercise 5: Performance Comparison

Insert 100,000 rows into a table. Compare query times with and without a JSONB GIN index.

Solutions

Solution 1

```
CREATE TABLE people (
  id SERIAL PRIMARY KEY,
  info JSONB NOT NULL
);
INSERT INTO people (info) VALUES
('{"name":"Alice","age":30,"job":"Engineer"}'),
('{"name":"Bob","hobbies":["chess","hiking"],"age":25}'),
('{"name":"Carol","address":{"city":"NYC"},"verified":true}'),
('{"name":"David","score":98.5}'),
('{"name":"Eve","languages":["Python","SQL","Go"]}');

SELECT info->>'name', info->>'age' FROM people WHERE (info->>'age')::INT > 25;
```

Solution 2

```
CREATE TABLE blog_posts (
  id SERIAL PRIMARY KEY,
  title TEXT,
  tags TEXT[]
);
INSERT INTO blog_posts (title, tags) VALUES
('Intro to PostgreSQL', ARRAY['postgresql','database','sql']),
('MongoDB vs PostgreSQL', ARRAY['mongodb','postgresql','nosql']),
('Redis Caching', ARRAY['redis','caching','performance']);

SELECT title FROM blog_posts WHERE 'postgresql' = ANY(tags);
```

Solution 4

```
CREATE TABLE employees (
  id SERIAL PRIMARY KEY,
  name VARCHAR(100),
  manager_id INT REFERENCES employees(id)
);
INSERT INTO employees (id, name, manager_id) VALUES
(1,'CEO',NULL),(2,'VP Eng',1),(3,'VP Sales',1),
```

```
(4, 'Sr Engineer', 2), (5, 'Engineer', 4), (6, 'Sales Rep', 3);

WITH RECURSIVE org AS (
  SELECT id, name, manager_id FROM employees WHERE id = 2
  UNION ALL
  SELECT e.id, e.name, e.manager_id FROM employees e
  JOIN org o ON e.manager_id = o.id
)
SELECT * FROM org;
```

Advanced Notes

PostgreSQL as a Multi-Model Database

PostgreSQL can serve as a multi-model database through:

- **JSONB:** Document store functionality
- **hstore:** Key-value store within a row
- **Arrays:** Multi-value columns
- **ltree:** Hierarchical/tree data
- **PostGIS:** Geospatial data
- **pg_vector:** Vector embeddings for AI/ML
- **TimescaleDB:** Time-series data
- **Recursive CTEs:** Graph traversal

This "PostgreSQL for everything" approach reduces operational complexity at the cost of some specialization.

The Impedance Mismatch Problem

Object-Oriented Programming languages use classes and objects; relational databases use tables and rows. This mismatch drives interest in document databases (which store objects natively as JSON). ORMs (Object-Relational Mappers) bridge this gap in RDBMS but add complexity. PostgreSQL's JSONB reduces this mismatch for hybrid data.

Cross-References

- **Previous:** [01 - What is a Database?](#)
- **Next:** [03 - Relational Databases](#)
- **Related:** [04 - OLTP vs OLAP](#)
- **Related:** [05 - CAP Theorem](#)
- **See Also:** [04 Advanced SQL/07 json jsonb.md](#) — Deep dive into PostgreSQL JSONB